Git Question Index - All 80 Questions

Q#	Question
Q1	What is version control? Centralized vs distributed
Q2	Git architecture - working directory, staging, local, remote
Q3	Git init, clone, config - setting up repos and identity
Q4	Git add, commit, status, log - basic workflow
Q5	Git diff - comparing working directory, staging, commits
Q6	Interactive staging (git add -p) - partial changes
Q7	Branches - create, switch, list, delete
Q8	Git merge - fast-forward, 3-way, merge commits
Q9	Git rebase - linear history, interactive, squashing
Q10	Rebase vs merge - when to use, golden rule
Q11	Git remote - origin, upstream, fetch, pull, push
Q12	Git pull vs fetch - pull --rebase
Q13	Git stash - save, apply, pop, list, drop
Q14	.gitignore - patterns, global, already-tracked files
Q15	Git tags - lightweight, annotated, semver
Q16	Git log - formatting, filtering, --oneline, --graph
Q17	HEAD, detached HEAD - meaning, recovery
Q18	Git reset - soft, mixed, hard
Q19	Git revert - safe undo without rewriting history
Q20	Git cherry-pick - apply specific commits
Q21	Git blame and bisect - who changed, when bugs appeared
Q22	Git clean and restore - untracked files, restoring

Q23	SSH keys and HTTPS - authentication
Q24	Pull requests / merge requests - code review
Q25	Git aliases and configuration - shortcuts
Q26	Branching strategies - Git Flow, GitHub Flow, trunk-based
Q27	Trunk-based development - short-lived branches, feature flags
Q28	Conventional commits - standards, changelog generation
Q29	Merge strategies - merge commit, squash, rebase merge
Q30	Git hooks - pre-commit, commit-msg, pre-push, server-side
Q31	Branch protection - reviews, status checks, signed commits
Q32	Git submodules - dependencies, updating, pitfalls
Q33	Git subtree - alternative to submodules
Q34	Monorepo vs polyrepo - trade-offs, tooling
Q35	Git LFS - binary files, storage, bandwidth
Q36	Git rebase --onto - advanced rebasing
Q37	Git reflog - recovering lost commits
Q38	Git worktree - multiple working directories
Q39	Signed commits and tags - GPG/SSH signing
Q40	Git secrets and security - scanning, hooks
Q41	GitHub Actions / GitLab CI - pipelines from Git events
Q42	Code owners - CODEOWNERS, auto review assignment
Q43	Release management - tagging, changelogs, semantic release
Q44	Git in CI/CD - shallow clone, sparse checkout, performance
Q45	Forking workflow - open source, upstream sync
Q46	GitHub vs GitLab vs Bitbucket - comparison

Q47	Git rerere - reuse recorded conflict resolution
Q48	Git patch and format-patch - sharing changes
Q49	Git internals - objects, SHA-1, packfiles
Q50	Garbage collection - gc, prune, pack, dangling
Q51	Monorepo at scale - sparse checkout, partial clone
Q52	Git server administration - bare repos, hooks, access
Q53	Platform migration - GitHub↔GitLab, SVN to Git
Q54	Git and compliance - audit trails, traceability
Q55	Rewriting history - filter-repo, BFG
Q56	Performance at scale - fsmonitor, maintenance
Q57	GitOps - Git as source of truth for infra
Q58	Merge conflict in critical production hotfix
Q59	Accidentally committed secrets to repo
Q60	Force push overwrote teammate's commits
Q61	Detached HEAD - commits lost
Q62	git reset --hard lost uncommitted work
Q63	Merge produced broken build - 3 days unnoticed
Q64	Repo is 10GB - clone takes 30 minutes
Q65	Rebase conflict hell - 50 conflicts
Q66	Wrong commit on main - revert vs reset
Q67	Git bisect finds bug 200 commits ago
Q68	Submodule stuck on wrong commit
Q69	CI fails only on main - works on feature branches
Q70	Committed to wrong branch - move commits

Q71	Deleted branch with unmerged work - recovery
Q72	200 tiny commits - clean before merge
Q73	Two devs rebased same branch - duplicates
Q74	Hotfix needed but main has unreleased features
Q75	.gitignore not working - file already tracked
Q76	Reverted merge commit, can't re-merge branch
Q77	Large binary committed - repo bloated permanently
Q78	Git pull creates unexpected merge commits
Q79	Feature branch 3 months old, 500 behind main
Q80	Same conflict appearing repeatedly

SECTION 1 - Junior Core Git (Q1-Q25)

Q1 What is version control? Centralized vs distributed

The Simple Answer

Version control tracks changes to files over time, enabling collaboration, history tracking, and rollback. It answers: who changed what, when, and why. Without version control, teams overwrite each other's work, cannot undo mistakes, and have no audit trail.

Centralized vs Distributed

Centralized (SVN, Perforce) - One central server holds the entire history. Developers check out files, make changes, commit back to the server. If the server is down, nobody can commit. If the server's disk fails and there is no backup, all history is lost.

Distributed (Git, Mercurial) - Every developer has a complete copy of the repository including full history. You can commit, branch, and view history offline. The "server" (GitHub, GitLab) is just a shared copy for collaboration - not a single point of failure. Every clone is a full backup.

> **Why Git won:** Distributed model (work offline, fast operations), branching is cheap (milliseconds), massive community, and GitHub/GitLab made collaboration frictionless. Git is used by 95%+ of developers worldwide.

Q2 Git architecture - working directory, staging, local, remote

The Three Trees

Working directory - Your filesystem. The files you see and edit. Contains tracked files (Git knows about) and untracked files (Git ignores).

Staging area (index) - A buffer between working directory and repository. git add moves changes from working directory to staging. Think of it as a "draft commit" - you curate exactly which changes go into the next commit.

Local repository (.git/) - The complete history of all commits, branches, and tags. Stored in the .git directory. git commit moves changes from staging to the local repo.

Remote repository - A shared copy on a server (GitHub, GitLab). git push sends local commits to the remote. git fetch downloads remote commits to local.

The Analogy - Writing a Book

> Working directory = your manuscript draft with edits. Staging area = the pages you have marked as "ready for the next chapter." Local repo = your personal bookshelf of completed chapters. Remote repo = the publisher's copy that your co-authors also access.

Q3 Git init, clone, config - setup

The Simple Answer

git init - Create a new Git repository in the current directory. Creates the .git/ directory. Used for starting a project from scratch.

git clone URL - Copy an existing remote repository to your machine. Downloads the full history. Sets up the remote origin automatically.

git config - Set user identity and preferences. git config --global user.name "Your Name" and git config --global user.email "you@example.com" identify you in commits. --global applies to all repos. Without --global, applies only to the current repo.

Essential Config

git config --global init.defaultBranch main - Use "main" instead of "master." git config --global pull.rebase true - Default to rebase on pull (cleaner history). git config --global core.editor vim - Set commit message editor.

Q4 Git add, commit, status, log - basic workflow

The Simple Answer

git status - Shows: modified files (not staged), staged files (ready to commit), untracked files (new). Run frequently - it is your orientation tool.

git add file - Stage a file for commit. git add . stages everything (use carefully). git add -A stages all changes including deletions.

git commit -m "message" - Save staged changes to the local repository with a descriptive message. Each commit gets a unique SHA-1 hash. Commits are permanent snapshots - you can always return to any commit.

git log - View commit history. Shows: commit hash, author, date, message. git log --oneline for compact view. git log --graph for visual branch structure.

Commit Message Best Practices

First line: imperative mood, under 50 characters: "Add user authentication." Body (after blank line): explain WHY, not what (the diff shows what). Reference issue/ticket numbers. Bad: "fixed stuff." Good: "Fix login timeout for users with special characters in password (#1234)."

Q5 Git diff - comparing changes

The Simple Answer

git diff - Shows unstaged changes (working directory vs staging area). **git diff --staged** - Shows staged changes (staging vs last commit). **git diff HEAD** - Shows all changes since last commit (staged + unstaged). **git diff branch1..branch2** - Shows differences between two branches. **git diff commit1..commit2** - Shows differences between two commits.

> **Reading diffs:** Lines starting with + are additions (green). Lines starting with - are removals (red). @@ markers show the location in the file. Learn to read diffs - code reviewers spend most of their time reading diffs.

Q6 Interactive staging (git add -p)

The Simple Answer

git add -p (patch mode) lets you stage individual hunks within a file rather than the entire file. You made 3 changes in one file but only want to commit 2 of them - add -p presents each hunk and asks: stage this hunk? (y)es, (n)o, (s)plit into smaller hunks, (e)dit manually.

Why It Matters

Professional commits are atomic - each commit contains exactly one logical change. If you modified a function AND fixed a typo in the same file, they should be two separate commits. git add -p enables this discipline without creating temporary files or stashing.

Q7 Branches - create, switch, list, delete

The Simple Answer

A branch is a pointer to a commit. Creating a branch is instantaneous - Git just creates a 41-byte file containing the commit hash. This is why Git branching is "cheap" compared to SVN (which copies the entire directory).

Commands

git branch feature-login - Create a branch. **git switch feature-login** - Switch to it (modern, replaces git checkout). **git switch -c feature-login** - Create and switch in one command. **git branch -d feature-login** - Delete a merged branch. **git branch -D feature-login** - Force delete an unmerged branch. **git branch -a** - List all branches (local and remote).

Naming Conventions

feature/login-page, bugfix/null-pointer, hotfix/security-patch, release/2.1.0. Use prefixes to categorize branches. Use slashes for hierarchy. Include ticket numbers: feature/JIRA-123-user-auth.

Q8 Git merge - fast-forward, 3-way, merge commits

Fast-Forward Merge

When the target branch has no new commits since the branch point, Git simply moves the pointer forward. No merge commit is created. Linear history. Happens when: you branch from main, do work, and main has not changed.

3-Way Merge

When both branches have diverged (both have new commits), Git creates a merge commit with two parents. The merge commit ties the two histories together. This is the default when fast-forward is not possible.

Merge Commit vs No Merge Commit

--ff-only - Only merge if fast-forward is possible. Fail otherwise. **--no-ff** - Always create a merge commit even if fast-forward is possible. Preserves branch history - you can see "this feature was developed on a branch." Many teams require --no-ff for feature branches to main.

Q9 Git rebase - linear history, interactive, squashing

The Simple Answer

Rebase moves your branch's commits on top of another branch. Instead of a merge commit, you get a linear history - as if you started your work from the latest commit on the target branch.

How It Works

git rebase main (while on feature branch): Git takes your feature commits, temporarily removes them, updates the branch to the tip of main, then re-applies your commits one by one on top. The result: your branch includes all of main's changes with your commits at the end.

Interactive Rebase

git rebase -i HEAD~5 - Rewrite the last 5 commits. Options per commit: pick (keep), reword (change message), squash (combine with previous), fixup (combine, discard message), drop (remove), edit (pause and amend). Used for cleaning up history before merging - combine WIP commits, fix typos in messages, reorder commits.

Squashing

Combine multiple commits into one. A branch with 15 commits ("WIP", "fix typo", "oops", "actually fix") becomes one clean commit: "Add user authentication feature." Use squash or fixup in interactive rebase, or use squash merge strategy.

Q10 Rebase vs merge - when to use, golden rule

When to Rebase

Updating a feature branch with the latest main: git rebase main - keeps your commits on top, clean linear history. Before merging to main: clean up your branch with interactive rebase. Personal branches that nobody else is using.

When to Merge

Merging a feature branch into main (preserves the branch history). When multiple people work on the same branch. When you need to preserve the exact history of how changes were integrated.

The Golden Rule of Rebasing

> **Never rebase a branch that other people are using.** Rebase rewrites commit hashes. If you rebase a shared branch, everyone else's local copy has the old hashes. When they pull, they get duplicate commits and a mess. Rebase your own branches. Merge shared branches. This rule prevents the nightmare described in scenario Q73.

Q11 Git remote - origin, upstream, fetch, push

The Simple Answer

origin - The default name for your primary remote (where you cloned from). git push origin main pushes to your remote. **upstream** - Convention for the original repository when you forked. Your fork is origin. The original is upstream. **git remote -v** - Show all remotes with URLs. **git remote add name URL** - Add a new remote.

Fetch vs Push

git fetch origin - Download commits from the remote but do not merge them. Your local branches are unchanged. Remote-tracking branches (origin/main) are updated. Safe - does not modify your work.

git push origin main - Upload local commits to the remote. Others can now see your changes.

Q12 Git pull vs fetch - pull --rebase

The Simple Answer

git fetch - Download remote changes. Does not modify your working directory or local branches. Just updates remote-tracking branches (origin/main).

git pull - Fetch + merge. Downloads remote changes AND merges them into your current branch. Can create merge commits if your branch and the remote have diverged.

git pull --rebase - Fetch + rebase. Downloads remote changes and rebases your local commits on top. Produces a linear history without merge commits. Recommended as the default: git config --global pull.rebase true.

> **Why pull --rebase is preferred:** git pull (without --rebase) creates a merge commit every time you sync with the remote - "Merge branch 'main' of github.com/..." These pollute the history with meaningless merge commits. pull --rebase keeps history clean. See scenario Q78.

Q13 Git stash - save, apply, pop, list

The Simple Answer

Stash saves your uncommitted changes temporarily and reverts your working directory to a clean state. Useful when: you need to switch branches but have uncommitted work, you need to pull but have local changes, or you want to temporarily set aside experiments.

Commands

git stash - Save all modified tracked files. **git stash -u** - Include untracked files. **git stash list** - Show all stashes. **git stash pop** - Apply the most recent stash and remove it. **git stash apply** - Apply without removing (keeps the stash). **git stash drop stash@{0}** - Delete a specific stash. **git stash show -p stash@{0}** - View stash contents.

> **Stash is not a substitute for commits.** If you stash frequently, you are probably avoiding commits. Commit work-in-progress on a feature branch instead - it is safer (stashes can be lost) and more descriptive.

Q14 .gitignore - patterns, already-tracked files

The Simple Answer

.gitignore tells Git which files to ignore - they will not appear in git status or be staged by git add .. Place .gitignore in the repository root. It is versioned (committed like any other file).

Common Patterns

*.log - All log files. node_modules/ - Node.js dependencies. .env - Environment variables. __pycache__/ - Python bytecode. .terraform/ - Terraform working directory. *.tfstate - Terraform state (should be in remote backend, not Git).

Already-Tracked Files

If a file is already tracked by Git, adding it to .gitignore does NOT stop tracking it. You must untrack it first: git rm --cached filename (removes from Git but keeps the file on disk). Then commit. Then .gitignore takes effect for future changes. This is the most common .gitignore confusion. See scenario Q75.

Global .gitignore

git config --global core.excludesfile ~/.gitignore_global. Add OS-specific files: .DS_Store (macOS), Thumbs.db (Windows), *~ (editor backups). These should not be in the project's .gitignore - they are developer-specific.

Q15 Git tags - lightweight, annotated, semver

The Simple Answer

Tags mark specific commits as important - typically releases. Unlike branches, tags do not move - they always point to the same commit.

Lightweight tag: git tag v1.0.0 - Just a pointer to a commit. No metadata. **Annotated tag:** git tag -a v1.0.0 -m "Release 1.0.0" - Includes tagger name, date, and message. Recommended for releases. **Push tags:** git push origin v1.0.0 or git push origin --tags (all tags). Tags are not pushed by default.

Semantic Versioning

MAJOR.MINOR.PATCH: v2.1.3. MAJOR = breaking changes. MINOR = new features, backward compatible. PATCH = bug fixes. Pre-release: v2.0.0-beta.1. Automated with tools like semantic-release (determines version from commit messages).

Q16 Git log - formatting, filtering

Useful Formats

git log --oneline - Short hash + first line of message. Compact. **git log --graph --oneline --all** - Visual branch structure. Essential for understanding branch topology. **git log --author="Alice"** - Filter by author. **git log --since="2025-01-01"** - Filter by date. **git log -- path/to/file** - History of a specific file. **git log -p** - Show the diff for each commit. **git log --stat** - Show files changed per commit.

Q17 HEAD, detached HEAD

The Simple Answer

HEAD is a pointer to the current commit - specifically, it points to the current branch, which points to a commit. When you switch branches, HEAD moves to the new branch.

Detached HEAD

When HEAD points directly to a commit (not a branch), you are in detached HEAD state. This happens when you: checkout a specific commit (git checkout abc1234), checkout a tag (git checkout v1.0.0), or during a rebase.

The Danger

Commits made in detached HEAD are not on any branch. If you switch to another branch, those commits become "orphaned" - still in the repository but not reachable from any branch. Eventually, garbage collection removes them. Recovery: git reflog shows the commits - create a branch from them: git branch recovery-branch abc1234. See scenario Q61.

Q18 Git reset - soft, mixed, hard

The Simple Answer

Reset moves the current branch pointer to a different commit. Three modes control what happens to the staging area and working directory.

--soft - Moves the branch pointer only. Staging area and working directory unchanged. The "undone" commits' changes appear as staged. Use for: combining the last 3 commits into one (reset --soft HEAD~3, then commit).

--mixed (default) - Moves the branch pointer and resets the staging area. Working directory unchanged. Changes appear as unstaged. Use for: unstaging files without losing changes.

--hard - Moves the branch pointer, resets staging, AND resets working directory. All changes are lost. Use for: completely discarding recent work. DANGEROUS - uncommitted changes are gone permanently (no reflog for unstaged changes).

> **The safety rule:** Use git stash before git reset --hard if there is any chance you need the changes. Or better: commit your work on a temporary branch before resetting.

Q19 Git revert - safe undo

The Simple Answer

git revert creates a new commit that undoes the changes from a specific commit. The original commit stays in history - nothing is rewritten. Safe for shared branches because it does not change existing commit hashes.

Revert vs Reset

Reset - Rewrites history. Removes commits. Dangerous on shared branches - other developers' history diverges.
Revert - Preserves history. Adds a new "undo" commit. Safe on shared branches - everyone pulls the revert commit normally.

The rule: Use revert for commits already pushed to shared branches. Use reset only for local, unpushed commits. This is the single most important undo decision in Git.

Q20 Git cherry-pick

The Simple Answer

git cherry-pick abc1234 takes a specific commit from another branch and applies it to the current branch. Creates a new commit with the same changes but a different hash.

Use Cases

Apply a bug fix from a feature branch to main without merging the entire feature. Backport a fix to a release branch. Pick specific commits from a branch that will not be merged entirely.

Caveats

Cherry-picked commits have different hashes than the originals. If the original branch is later merged, Git may flag the same change as a conflict. Use sparingly - prefer merging entire branches when possible.

Q21 Git blame and bisect

git blame

git blame file shows who last modified each line of a file. Useful for: understanding who wrote code, finding when a line was changed, and identifying the commit that introduced a specific change. git blame -L 10,20 file - blame only lines 10-20.

git bisect

Binary search through commit history to find which commit introduced a bug. git bisect start, git bisect bad (current commit is broken), git bisect good v1.0.0 (this commit was working). Git checks out the middle commit - you test and mark good or bad. Repeat. Finds the exact commit in log2(n) steps - 200 commits searched in 8 steps. Can be automated: git bisect run ./test_script.sh.

Q22 Git clean and restore

git clean

Removes untracked files from the working directory. **git clean -n** - Dry run (show what would be removed). **git clean -f** - Force remove untracked files. **git clean -fd** - Remove untracked files and directories. **git clean -fX** - Remove only ignored files. Useful for cleaning build artifacts.

git restore (Git 2.23+)

Modern replacement for git checkout for file restoration. **git restore file** - Discard changes in working directory (restore from staging area). **git restore --staged file** - Unstage a file (opposite of git add). **git restore --source=HEAD~3 file** - Restore a file from 3 commits ago. Clearer than git checkout which was overloaded (switch branches AND restore files).

Q23 SSH keys and HTTPS - authentication

SSH

Generate a key pair: ssh-keygen -t ed25519. Add the public key to GitHub/GitLab. Clone with: git clone git@github.com:user/repo.git. Authenticate automatically without passwords. Recommended for developers who push/pull frequently.

HTTPS

Clone with: git clone https://github.com/user/repo.git. Authenticate with username/password or personal access token. Git credential helpers cache credentials: git config --global credential.helper cache (in-memory) or store (on disk). GitHub no longer accepts passwords - use personal access tokens or SSH keys.

> **For CI/CD:** Use personal access tokens (PATs) or deploy keys (read-only SSH keys). For GitHub Actions: the built-in GITHUB_TOKEN works for the current repository.

Q24 Pull requests / merge requests - code review

The Simple Answer

A PR (GitHub/Bitbucket) or MR (GitLab) is a request to merge changes from one branch into another. It provides: a diff of all changes, a discussion thread for code review, CI/CD status checks, and approval workflow.

The Review Process

Developer pushes feature branch → opens PR → CI runs tests → reviewers examine the diff → reviewers approve or request changes → developer addresses feedback → PR is merged. The PR becomes a permanent record of why changes were made, who reviewed them, and what discussions occurred.

Best Practices

Small PRs (under 400 lines). Descriptive title and description explaining the WHY. Link to the issue/ticket. Self-review before requesting others. Respond to all comments. At least one approval required before merge.

Q25 Git aliases and configuration

The Simple Answer

Aliases create shortcuts for frequently used commands. git config --global alias.co checkout - now git co works. git config --global alias.st status. git config --global alias.lg "log --oneline --graph --all" - visual branch log.

Productivity Aliases

alias.unstage = "reset HEAD --". alias.amend = "commit --amend --no-edit". alias.last = "log -1 HEAD". alias.branches = "branch -a --sort=-committerdate" (branches sorted by recent activity).

SECTION 2 - Mid-Level Workflows & Production (Q26-Q48)

Q26 Branching strategies - Git Flow, GitHub Flow, trunk-based

Git Flow

Long-lived branches: main (production), develop (integration). Feature branches from develop. Release branches from develop for release prep. Hotfix branches from main for urgent fixes. Complex but structured. Best for: software with scheduled releases and multiple supported versions.

GitHub Flow

Simple: main + feature branches. Branch from main, develop, open PR, review, merge to main, deploy. Every merge to main is deployable. Best for: continuous deployment, web applications, SaaS products.

GitLab Flow

Extension of GitHub Flow. Adds environment branches: main → staging → production. Or release branches: main → release/1.0 → release/2.0. Best for: teams needing environment-based or release-based deployment.

Trunk-Based Development

Everyone commits to main (trunk) directly or via very short-lived branches (hours, not days). Feature flags hide incomplete work. Requires: strong CI, comprehensive tests, feature flag infrastructure. Best for: high-performing teams practicing continuous deployment. See Q27.

> **The modern trend:** Teams are moving from Git Flow (complex, slow) to trunk-based development (simple, fast). GitHub Flow is the practical middle ground for most teams.

Q27 Trunk-based development - feature flags

The Simple Answer

Trunk-based development means all developers commit to a single branch (main/trunk). Branches are short-lived (less than 24 hours). Code is always in a deployable state on main. Incomplete features are hidden behind feature flags, not long-running branches.

Feature Flags

A conditional in code that enables or disables functionality: if (featureFlags.isEnabled('new-checkout')) { newCheckout(); } else { oldCheckout(); }. The flag is controlled externally (LaunchDarkly, Unleash, config file). Deployment and release are decoupled - you deploy code with the flag off, then enable it for specific users, regions, or percentages.

Why It Is Superior

No merge conflicts from long-running branches. Smaller, more frequent commits. Issues are caught immediately by CI. Code reviews are smaller and faster. Features can be released to 1% of users (canary) before full rollout.

Q28 Conventional commits - standards

The Simple Answer

A standardized commit message format: type(scope): description. Types: feat (new feature), fix (bug fix), docs, style, refactor, test, chore, ci. Examples: feat(auth): add SSO login, fix(api): handle null response, docs(readme): update install instructions.

Why It Matters

Automated changelog generation. Automatic semantic versioning (feat = minor bump, fix = patch bump, BREAKING CHANGE = major bump). Tools: commitlint (validates messages), semantic-release (automates versioning), standard-version (changelog generation).

Q29 Merge strategies - merge commit, squash, rebase merge

Merge Commit (--no-ff)

Creates a merge commit with two parents. Preserves full branch history. You can see all individual commits from the feature branch. Best when: branch history is meaningful and clean.

Squash Merge

Combines all branch commits into a single commit on the target branch. Clean history - one commit per feature. Branch history is lost. Best when: the branch has messy WIP commits and you want one clean commit per feature.

Rebase Merge

Replays branch commits on top of the target branch. No merge commit. Linear history. Each commit is preserved individually. Best when: branch commits are clean and individually meaningful.

> **Team decision:** Pick one strategy and enforce it across the team. Most teams use squash merge (simplest, cleanest) or merge commits (most complete history). Rebase merge is the least common because it requires clean branch history.

Q30 Git hooks - pre-commit, commit-msg, pre-push

Client-Side Hooks

pre-commit - Runs before commit is created. Use for: linting, formatting, secret scanning (gitleaks). If the hook exits non-zero, the commit is aborted. **commit-msg** - Validates the commit message. Enforce conventional commits format. **pre-push** - Runs before pushing. Use for: running tests, checking branch naming conventions.

Server-Side Hooks

pre-receive - Runs on the server before accepting a push. Enforce branch protection, reject force pushes, validate commit signatures. **post-receive** - Runs after a push is accepted. Trigger CI/CD, send notifications.

pre-commit Framework

Install via pip install pre-commit. Define hooks in .pre-commit-config.yaml. pre-commit install installs hooks for all developers. Runs automatically on every commit. Supports hooks from the community: gitleaks (secrets), black (Python formatting), eslint (JavaScript), yamllint.

Q31 Branch protection - reviews, status checks

The Simple Answer

Branch protection rules prevent direct pushes to critical branches (main, release). Enforced by the Git platform (GitHub, GitLab, Bitbucket).

Common Rules

Required pull request reviews - At least 1-2 approvals before merge. Dismiss stale approvals (require re-review after new commits). **Required status checks** - CI must pass before merge. Specify which checks are required. **Require signed commits** - Only GPG/SSH-signed commits allowed. **No force pushes** - Prevent history rewriting

on protected branches. **Require linear history** - Force squash or rebase merge (no merge commits). **Include administrators** - Even admins must follow the rules.

Q32 Git submodules

The Simple Answer

Submodules embed one Git repository inside another. The parent repo tracks a specific commit of the submodule. Used for: shared libraries, common configuration, third-party dependencies with custom patches.

Common Problems

Submodule stuck on old commit - The parent repo pins the submodule to a commit. Updating requires: cd submodule, git pull, cd .., git add submodule, git commit. Developers forget this - the submodule stays on the old version. See scenario Q68.

Clone does not include submodules - git clone alone does not download submodule content. Need: git clone --recurse-submodules or git submodule update --init --recursive after cloning.

> **The honest opinion:** Submodules are confusing and error-prone. Consider alternatives: Git subtree (simpler), package managers (npm, pip - designed for dependency management), or monorepo (eliminate the need for cross-repo dependencies). Use submodules only when no better option exists.

Q33 Git subtree - alternative to submodules

The Simple Answer

Git subtree merges another repository into a subdirectory of your repository. Unlike submodules, the subtree content is part of your repo - no special commands needed for cloning, no .gitmodules file, and developers do not need to know about the subtree.

How It Works

git subtree add --prefix=libs/shared https://github.com/org/shared.git main - Adds the shared repo into libs/shared/. Updates: git subtree pull --prefix=libs/shared https://github.com/org/shared.git main. Contributing back: git subtree push.

Subtree vs Submodule

Feature	Submodule	Subtree
Content in parent repo	No (pointer only)	Yes (full copy)
Clone complexity	Extra commands needed	Just git clone
Update complexity	cd + pull + add + commit	subtree pull
History	Separate (own repo)	Merged into parent
Developer awareness	Must know about submodules	Transparent

Q34 Monorepo vs polyrepo

Monorepo

All projects in one repository. Google, Meta, and many large companies use monorepos. Advantages: atomic cross-project changes, single version of truth, easier code sharing, unified CI/CD. Disadvantages: large repo size, slow operations without tooling, everyone sees everything, complex CI filtering.

Polyrepo

Each project/service in its own repository. Advantages: clear ownership, independent versioning, smaller repos, simple permissions. Disadvantages: cross-repo changes require multiple PRs, dependency management is complex, inconsistent tooling.

Monorepo Tooling

Nx - Smart build system. Only builds/tests affected projects. **Turborepo** - Fast, caching build system for JavaScript monorepos. **Bazel** - Google's build system. Language-agnostic. Powerful but complex.

Q35 Git LFS - large file storage

The Simple Answer

Git is designed for text files. Large binary files (images, videos, models, datasets) bloat the repository because Git stores every version of every file. Git LFS stores large files on a separate server and replaces them with lightweight pointers in the repository.

Setup

git lfs install. git lfs track "*.psd" (track PSD files). This creates .gitattributes. Commit and push normally - LFS handles large file storage transparently. git lfs ls-files lists tracked files.

Limitations

LFS requires server support (GitHub, GitLab, Bitbucket all support it). Storage and bandwidth limits apply (GitHub: 1 GB free, then $5/50GB). Clone still downloads all LFS objects by default - use git lfs fetch --recent for partial downloads.

Q36 Git rebase --onto

The Simple Answer

rebase --onto moves a range of commits to a new base. git rebase --onto new-base old-base branch - Take commits from old-base to branch and replay them onto new-base. Use case: you branched from feature-A, but feature-A is not ready. You want to move your commits to main instead: git rebase --onto main feature-A your-branch.

Q37 Git reflog - recovering lost commits

The Simple Answer

Reflog records every time HEAD changes - every commit, checkout, merge, rebase, reset, and pull. It is your safety net when things go wrong. git reflog shows a chronological list of where HEAD has been. Even after git reset --hard, the "lost" commits are still in the reflog for 90 days (default).

Recovery Pattern

Mistake: git reset --hard HEAD~5 (accidentally removed 5 commits). Recovery: git reflog shows the commit before the reset. git reset --hard abc1234 (the commit hash from reflog) restores everything.

Reflog is local only. It is not pushed to remotes. It exists only on the machine where the action happened. And it expires - default 90 days for reachable, 30 days for unreachable entries. Recover quickly.

Q38 Git worktree

The Simple Answer

Worktree lets you check out multiple branches simultaneously in different directories - without cloning the repo again. git worktree add ../hotfix-branch hotfix/urgent creates a new directory with the hotfix branch checked out. You can work on main in the original directory and the hotfix in the new directory simultaneously.

Use Cases

Working on a hotfix while keeping your feature branch work intact. Running tests on one branch while developing on another. Comparing behavior between branches side by side. Reviewing a PR while working on your own code.

Q39 Signed commits and tags

The Simple Answer

Signing proves that a commit was actually made by the claimed author. Without signing, anyone can commit as any name/email - git config user.email "ceo@company.com" works. Signing uses GPG or SSH keys to cryptographically verify the author.

Setup

GPG: git config --global commit.gpgsign true, git config --global user.signingkey YOUR_GPG_KEY_ID. SSH (Git 2.34+): git config --global gpg.format ssh, git config --global user.signingkey ~/.ssh/id_ed25519.pub.

Verification

GitHub/GitLab show a "Verified" badge on signed commits. Required by some compliance frameworks. Branch protection can require signed commits - unsigned commits are rejected.

Q40 Git secrets and security

The Simple Answer

Secrets in Git repos are a top security risk. Once committed, they are in the history forever - even if deleted in a subsequent commit. Secrets in public repos are found by automated scanners within minutes.

Prevention

Pre-commit scanning: gitleaks, git-secrets, trufflehog run before each commit. Block commits containing patterns matching AWS keys, passwords, private keys. **GitHub secret scanning:** Automatically scans repos and alerts on detected secrets. Partners (AWS, GCP, Slack) are notified and revoke compromised tokens. **Repository scanning:** Regular scans of the entire history, not just new commits.

Q41 GitHub Actions / GitLab CI integration

The Simple Answer

Git events (push, pull request, tag) trigger CI/CD pipelines. The pipeline definition lives in the repository (workflow files for Actions, .gitlab-ci.yml for GitLab).

GitHub Actions Key Concepts

Workflows (.github/workflows/*.yml), jobs (run in parallel by default), steps (sequential within a job), actions (reusable components). Triggers: push, pull_request, schedule, workflow_dispatch (manual), release.

GitLab CI Key Concepts

.gitlab-ci.yml in repo root. Stages (build, test, deploy), jobs (assigned to stages), runners (execute jobs). Pipelines trigger on push. Merge request pipelines run on MR creation/update. Environment-specific deployments with manual approval.

Q42 Code owners

The Simple Answer

CODEOWNERS file maps file paths to responsible teams/individuals. When a PR modifies files in a path, the corresponding owners are automatically added as required reviewers. *.js @frontend-team, /infrastructure/ @devops-team, /docs/ @technical-writers. Ensures the right people review every change.

Q43 Release management - tagging, changelogs

The Simple Answer

Tagging: Tag every release: git tag -a v2.1.0 -m "Release 2.1.0". Push: git push origin v2.1.0. The tag is the permanent record of exactly which commit was released.

Changelogs: Generated from conventional commits. semantic-release automates: determine next version from commits → generate changelog → create Git tag → publish release. Removes manual versioning entirely.

GitHub Releases: Attach binaries, release notes, and changelogs to a Git tag. Create via the GitHub UI or API. Integrates with GitHub Actions for automated release creation.

Q44 Git in CI/CD - shallow clone, sparse checkout, performance

Shallow Clone

git clone --depth=1 downloads only the latest commit - no history. Dramatically faster for large repos. Used in CI where full history is not needed. Limitation: git log and git blame do not work beyond the shallow depth. Use --depth=50 if you need recent history for changelogs.

Sparse Checkout

Download only specific directories: git sparse-checkout set path/to/project. Combined with partial clone (--filter=blob:none): only download file content when accessed. Essential for monorepos in CI - do not download the entire repo when you only build one project.

CI Performance Tips

Cache .git directory between runs. Use shallow clones. Use Git protocol v2 (faster negotiation). Fetch only the branch being built: git fetch origin pull/123/head --depth=1.

Q45 Forking workflow

The Simple Answer

Fork the original repo to your account. Clone YOUR fork. Create a feature branch. Push to your fork. Open a PR from your fork to the original repo. Maintainers review and merge.

Keeping Your Fork Updated

git remote add upstream https://github.com/original/repo.git. git fetch upstream. git rebase upstream/main. git push origin main. This syncs your fork with the original without creating merge commits.

Q46 GitHub vs GitLab vs Bitbucket

Comparison

Feature	GitHub	GitLab	Bitbucket
Market share	Largest (100M+ users)	Second largest	Third (Atlassian)
CI/CD	Actions (excellent)	Built-in (excellent)	Pipelines (good)
Self-hosted	Enterprise ($$$)	Free (CE)	Data Center ($$)
Best for	Open source, startups	DevOps platform	Atlassian shops
Unique features	Copilot, Sponsors	Full DevOps lifecycle	Jira integration

The decision: GitHub for open source, community, and AI features. GitLab for self-hosted, all-in-one DevOps platform. Bitbucket for Atlassian ecosystem (Jira, Confluence). Most teams choose GitHub unless they need self-hosted (GitLab) or are deeply invested in Atlassian.

Q47 Git rerere - reuse recorded resolution

The Simple Answer

Git rerere (reuse recorded resolution) remembers how you resolved a merge conflict and automatically applies the same resolution if the same conflict appears again. Enable: git config --global rerere.enabled true.

When It Helps

Rebasing a long-running branch repeatedly - the same conflicts appear each time you rebase. With rerere, you resolve once and Git applies the resolution automatically on subsequent rebases. Also useful when cherry-picking the same commits across multiple branches. See scenario Q80.

Q48 Git patch and format-patch

The Simple Answer

git format-patch - Creates patch files from commits. git format-patch -3 creates patches for the last 3 commits. Each patch is an email-formatted file containing the diff, commit message, and author info.

git am - Applies patches. git am *.patch applies patch files as commits, preserving authorship and messages.

Use Cases

Sharing changes with teams who do not have push access. Email-based code review (Linux kernel development). Applying commits across disconnected repositories. Creating a portable record of changes.

SECTION 3 - Senior Advanced & Scale (Q49-Q57)

Q49 Git internals - objects, SHA-1, packfiles

The Simple Answer

Git is a content-addressable filesystem. Everything is stored as objects identified by SHA-1 hashes. Understanding internals helps debug unusual situations and optimize performance.

Object Types

Blob - File content. No filename, no metadata - just the raw content. Two files with identical content share the same blob.

Tree - Directory listing. Maps filenames to blob hashes (files) and other tree hashes (subdirectories). A tree is a snapshot of a directory at a point in time.

Commit - Points to a tree (the project snapshot), parent commit(s), author, committer, timestamp, and message. A commit is a snapshot of the entire project plus metadata.

Tag - Points to a commit (or any object) with a name, tagger, and message. Only annotated tags are objects - lightweight tags are just refs.

SHA-1 Hashing

Every object is identified by the SHA-1 hash of its content. The hash guarantees: the same content always produces the same hash, and any change produces a completely different hash. This is how Git detects corruption and deduplicates content. Git is transitioning to SHA-256 for stronger cryptographic properties.

Packfiles

Initially, Git stores each object as a separate file (loose objects). Periodically (or during git gc), Git compresses related objects into packfiles using delta compression - storing only the differences between similar objects. This dramatically reduces storage for repositories with many versions of the same files.

Exploring Internals

git cat-file -t abc1234 - Show object type (blob, tree, commit, tag). git cat-file -p abc1234 - Pretty-print object content. git rev-parse HEAD - Show the SHA-1 of HEAD. git count-objects -v - Show object storage statistics. find .git/objects -type f - See loose objects on disk.

Q50 Garbage collection - gc, prune, dangling objects

The Simple Answer

Git accumulates loose objects over time - abandoned commits from rebases, reset, amended commits, and deleted branches. Garbage collection cleans up these unreachable objects.

git gc - Packs loose objects into packfiles, prunes unreachable objects older than 2 weeks (default), and optimizes the repository. Runs automatically after certain operations (push, merge). **git prune** - Removes unreachable objects immediately (dangerous - bypasses the 2-week safety window). **git fsck** - Checks repository integrity. Finds dangling (unreachable) objects, corrupt objects, and other issues.

Dangling Objects

A dangling commit is one not reachable from any branch or tag. Created by: rebasing (old commits become dangling), reset --hard (abandoned commits), deleted branches with unmerged commits. They are recoverable via reflog until gc removes them (default: 30 days for unreachable, 90 days for reachable).

The safety net: Git's conservative gc defaults (2-week prune delay, 90-day reflog) protect against accidental data loss. Do not run git prune manually unless you understand the consequences. git gc is safe and should be run periodically on large repos.

Q51 Monorepo at scale - sparse checkout, partial clone

The Challenge

A monorepo with 100 projects, 500,000 commits, and 50 GB of data. Full clone takes 30+ minutes. Every git status checks the entire working directory. CI builds everything for every change.

Sparse Checkout

Download the full .git history but only check out specific directories: git sparse-checkout init --cone, git sparse-checkout set project-a project-b. Your working directory contains only project-a/ and project-b/. Everything else exists in Git history but is not on disk. Dramatically reduces working directory size and git status speed.

Partial Clone

git clone --filter=blob:none URL - Downloads commits and trees but NOT file content (blobs). Blobs are downloaded on demand when you checkout or access them. Combined with sparse checkout: only the blobs for your sparse-checked-out files are ever downloaded. Reduces initial clone from 50 GB to potentially under 1 GB.

Affected Targets

Tools like Nx, Turborepo, and Bazel analyze which projects are affected by a change. CI only builds and tests affected projects: if you changed project-a, only build project-a and its dependents, not the other 99 projects. nx affected:test runs tests only for affected projects.

Git Maintenance

git maintenance start enables background optimization: prefetch (download remote objects proactively), gc (periodic garbage collection), commit-graph (speed up graph traversal). Essential for developer productivity on large repos.

Q52 Git server administration - bare repos, hooks, access

Bare Repositories

A bare repo has no working directory - only the .git contents. Used for: shared servers, central repositories. Created with git init --bare. Push/pull to bare repos. Never commit directly to them.

Server-Side Hooks

pre-receive - Runs before accepting a push. Use for: enforcing branch naming conventions, requiring signed commits, blocking force pushes to protected branches, rejecting large files, validating commit messages. If the hook exits non-zero, the entire push is rejected.

post-receive - Runs after a push is accepted. Use for: triggering CI/CD pipelines, sending notifications, updating deployment, mirroring to other repositories.

Access Control

Git itself has no built-in access control. Platforms (GitHub, GitLab, Gitolite) add: per-repository permissions (read, write, admin), per-branch permissions (who can push to main), RBAC for teams and organizations.

Q53 Platform migration - SVN to Git, GitHub to GitLab

SVN to Git

git svn clone - Converts SVN history to Git commits. Preserves history, authors, and branches. Author mapping file converts SVN usernames to Git author format. Large SVN repos may take hours to convert. Post-migration: verify history, tag releases, set up CI/CD, update developer workflows.

GitHub to GitLab (or reverse)

Repository: git clone --mirror from source, git push --mirror to destination. Preserves all branches, tags, and history. **Issues/PRs:** Use migration tools (GitLab import, GitHub Import API). Not all metadata transfers cleanly (comments, reactions, labels may need manual adjustment). **CI/CD:** Rewrite pipeline definitions (.github/workflows/ to .gitlab-ci.yml or vice versa). Different syntax, different concepts.

Key Considerations

Update all developer remotes (git remote set-url origin new-url). Update CI/CD webhooks. Update internal documentation and links. Run both platforms in parallel for 2-4 weeks. Communicate the cutover date clearly.

Q54 Git and compliance - audit trails, traceability

The Simple Answer

Regulated industries require traceability: who changed what, when, why, and who approved it. Git provides this when configured properly.

Compliance Controls

Signed commits - Cryptographic proof of author identity. Required by: SOC 2 Type II, PCI DSS, FedRAMP. **Branch protection** - Enforce review requirements. At least 2 reviewers for production changes. No direct pushes. **PR/MR as audit trail** - Every change has: description (why), diff (what), reviewers (who approved), CI results (verified). **Immutable history** - No force pushes to protected branches. No history rewriting on main.

Traceability

Commit messages reference issue/ticket numbers. PRs link to change requests. Tags mark releases. git log --grep="JIRA-123" finds all commits for a ticket. The chain: Requirement → Ticket → PR → Commits → Deployment → Release provides end-to-end traceability required by auditors.

Q55 Rewriting history - filter-repo, BFG

The Simple Answer

Sometimes you need to rewrite Git history: remove accidentally committed secrets, delete large binary files that bloated the repo, or change author information across all commits.

git filter-repo (Recommended)

The modern replacement for git filter-branch (deprecated - too slow and error-prone). git filter-repo --invert-paths --path secrets.env - Remove a file from all history. git filter-repo --replace-text expressions.txt - Replace text across all commits. Fast, safe, and well-documented.

BFG Repo Cleaner

Simpler for common tasks. bfg --delete-files *.mp4 - Remove all MP4 files from history. bfg --replace-text passwords.txt - Replace secrets with ***REMOVED***. Faster than filter-repo for large repos. Does not modify the latest commit - only history.

After Rewriting

Force push all branches: git push --force --all. All developers must re-clone or reset their local repos - their history no longer matches the remote. Verify with git log and git fsck. Update any CI/CD caches.

> **Warning:** History rewriting is a team-wide disruptive event. Coordinate with all developers. Schedule during low-activity periods. Communicate clearly. Consider whether the disruption is worth it - for secrets, yes (they must be removed). For cleanup, maybe not.

Q56 Performance at scale - fsmonitor, maintenance

The Simple Answer

Large repos (millions of files, hundreds of thousands of commits) make Git operations slow. Modern Git has features to address this.

fsmonitor

Git normally scans the entire working directory to determine file status. fsmonitor uses the OS file system watcher (FSEvent on macOS, inotify on Linux, ReadDirectoryChangesW on Windows) to track which files changed. git status becomes instant instead of scanning every file. Enable: git config core.fsmonitor true (Git 2.37+).

Commit Graph

git commit-graph write creates a binary file that accelerates graph traversal operations (log, merge-base, reachability checks). Dramatically speeds up git log --graph and git merge-base on repos with many commits.

Multi-Pack Index

Indexes across multiple packfiles - speeds up object lookup in repos with many packfiles. Enabled automatically by git gc in modern Git.

git maintenance

Background optimization: git maintenance start. Runs: prefetch (download remote objects periodically), gc (pack objects), commit-graph (update graph), loose-objects (pack loose objects). Runs automatically in the background. Essential for large repo performance.

Q57 GitOps - Git as source of truth

The Simple Answer

GitOps uses Git as the single source of truth for infrastructure and application deployments. All changes go through Git (commit → PR → review → merge). An operator (ArgoCD, Flux) watches the Git repository and applies changes to the target environment automatically.

The Pattern

Developer commits a Kubernetes manifest change to Git → PR is reviewed and approved → Merge to main → ArgoCD detects the change → ArgoCD applies the manifest to the cluster → The cluster state matches Git. If someone modifies the cluster directly, ArgoCD detects the drift and reverts to the Git state.

Key Principles

Declarative - Desired state is described in Git (YAML manifests, Helm values, Kustomize overlays), not imperative commands. **Versioned and auditable** - Every change is a Git commit with author, timestamp, message, and reviewer. Rollback = revert the Git commit. **Automatically applied** - The operator continuously reconciles cluster state with Git. No manual kubectl apply. **Observable** - The operator reports sync status. Alerts on drift.

ArgoCD vs Flux

ArgoCD - Rich UI. Multi-cluster support. Application-centric. More popular. **Flux** - Lighter weight. CLI-first. Better for simple setups. Native Kustomize and Helm support. Both are CNCF projects. ArgoCD is more common in enterprise environments.

SECTION 4 - Tricky Scenarios (Q58-Q80)

Q58 Merge conflict in production hotfix

Step-by-Step

Step 1 - Stay calm. Merge conflicts are normal. They mean two people changed the same lines - Git needs human judgment to decide which changes to keep.

Step 2 - Understand the conflict markers. <<<<<<< HEAD shows your changes. ======= separates the two versions. >>>>>>> branch-name shows the incoming changes. Edit the file to keep the correct code - this might be one side, the other, or a combination.

Step 3 - For a hotfix under pressure. Focus on the conflicting files. Understand both changes. If unsure, talk to the other author. Do not blindly accept one side - a wrong resolution can introduce bugs.

Step 4 - Resolve, test, merge. Edit the conflicting files. Remove conflict markers. git add the resolved files. Run tests. git commit to complete the merge. Push immediately for the hotfix.

Step 5 - If the conflict is too complex, abort: git merge --abort. This restores the pre-merge state. Take time to understand the changes, then retry.

Q59 Accidentally committed secrets to repo

Step-by-Step

Step 1 - Rotate the secret immediately. Assume it is compromised. Change the password, regenerate the API key, revoke the token. Do this BEFORE cleaning the Git history - attackers scan repos in real-time.

Step 2 - Remove from history. git filter-repo --invert-paths --path config/secrets.yml removes the file from all history. Or BFG: bfg --delete-files secrets.yml. Force push all branches: git push --force --all.

Step 3 - If on GitHub, contact GitHub support to purge cached copies. Enable GitHub secret scanning to catch future leaks.

Step 4 - Notify teammates. Everyone must re-clone or reset their local repos. Old history in local repos still contains the secret.

Step 5 - Prevent. Pre-commit hooks with gitleaks or git-secrets. .gitignore for secret files. Use environment variables or secret managers instead of files.

Q60 Force push overwrote teammate's commits

What Happened

You rebased your branch and force pushed. Your teammate had pushed commits to the same branch. Your force push overwrote their commits - they are gone from the remote.

Step-by-Step Recovery

Step 1 - The teammate's local repo still has their commits. Have them push their branch to a temporary name: git push origin HEAD:rescue/their-work.

Step 2 - Check the remote reflog. Some platforms (GitLab) keep a server-side reflog. GitHub does not - recovery depends on someone's local repo.

Step 3 - Merge the rescued commits. Cherry-pick or merge the teammate's commits into the rebased branch.

Prevention

Never force push to a shared branch. Use --force-with-lease instead of --force - it fails if the remote has commits you have not seen, preventing overwrites. Configure branch protection to block force pushes on shared branches. Communicate before rebasing any branch others might be using.

Q61 Detached HEAD - commits lost

What Happened

You checked out a tag or specific commit. Made several commits. Switched to main. Now those commits seem lost - they are not on any branch.

Recovery

Step 1 - git reflog. Shows where HEAD has been. Find the commit hash of your last commit in detached HEAD.

Step 2 - Create a branch. git branch recovery abc1234 (using the hash from reflog). Now your commits are on the recovery branch.

Step 3 - Merge or cherry-pick the recovery branch into your target branch.

Prevention

Git warns when you enter detached HEAD state. If you intend to make commits, create a branch first: git switch -c new-branch. If you accidentally committed in detached HEAD, do not panic - reflog has your commits for 30 days.

Q62 git reset --hard lost uncommitted work

The Harsh Reality

git reset --hard discards uncommitted changes in the working directory and staging area. These changes were NEVER committed - they are not in the reflog. Reflog only tracks committed changes.

Recovery Attempts

Staged changes (git add was run): git fsck --lost-found may recover blobs from the staging area. Check .git/lost-found/other/ for recovered blobs. This is not guaranteed.

Unstaged changes: Gone. No Git recovery possible. Check: IDE local history (VS Code, IntelliJ save previous versions), filesystem snapshots (Time Machine on macOS, LVM snapshots on Linux), backup systems.

Prevention

Commit frequently. Even "WIP" commits on a branch are recoverable via reflog. Uncommitted work is at risk. Use git stash as a safety step before any destructive operation. Configure your IDE to save local history automatically.

Q63 Merge produced broken build - unnoticed 3 days

What Happened

A merge to main passed CI (tests passed) but introduced a subtle bug - a race condition that only appears under load. Production has been running broken code for 3 days.

Step-by-Step

Step 1 - Revert the merge immediately. git revert -m 1 merge_commit_hash. This undoes the merge without rewriting history. Deploy the revert.

Step 2 - Identify the issue. Review the merge diff. Run integration tests. Load test the merged code.

Step 3 - Fix and re-merge. Fix the bug on the feature branch. Create a new PR. Add tests that catch the specific issue. Re-merge.

Prevention

Automated tests must cover critical paths. Add integration tests and load tests to CI. Post-deployment monitoring should catch regressions (error rate, latency). Canary deployments limit blast radius - if the first 5% of traffic shows issues, the full deployment is halted.

Q64 Repo is 10GB - clone takes 30 minutes

Step-by-Step

Step 1 - Diagnose what is consuming space. git rev-list --objects --all | git cat-file --batch-check='%(objecttype) %(objectname) %(objectsize) %(rest)' | sort -k3 -n -r | head -20 shows the largest objects. Usually: large binary files (videos, datasets, build artifacts) accidentally committed.

Step 2 - Remove large files from history. BFG: bfg --strip-blobs-bigger-than 10M. Or filter-repo: git filter-repo --strip-blobs-bigger-than 10M. Force push. All developers re-clone.

Step 3 - Prevent future bloat. Set up Git LFS for binary files. Add binary patterns to .gitignore. Pre-receive hook rejecting files over 5 MB. git config --global core.bigFileThreshold 1m warns developers about large files.

Step 4 - For CI/CD, use shallow clone: git clone --depth=1.

Q65 Rebase conflict hell - 50 conflicts

What Happened

A feature branch has been open for 3 months. Main has diverged significantly. git rebase main produces 50+ conflicts across dozens of files, one commit at a time.

Step-by-Step

Step 1 - Abort if overwhelmed. git rebase --abort returns to the pre-rebase state. Nothing is lost.

Step 2 - Consider merge instead. git merge main produces ONE conflict resolution (all conflicts at once) instead of commit-by-commit. Easier when there are many small commits with overlapping conflicts. The merge commit is less "clean" than a rebase but much faster to resolve.

Step 3 - If rebase is required, enable rerere first: git config rerere.enabled true. Resolve conflicts once - rerere remembers the resolution for subsequent conflicts with the same content.

Step 4 - Interactive rebase to squash first. Squash your 50 commits into 5 logical ones. Then rebase - 5 potential conflict points instead of 50.

Prevention

Rebase onto main frequently (daily or at least weekly). Short-lived branches (days, not months). Trunk-based development with feature flags eliminates long-running branches entirely. See scenario Q79.

Q66 Wrong commit on main - revert vs reset

The Decision

If the commit is already pushed and others have pulled it: Use git revert. Creates a new undo commit. Safe. Everyone pulls the revert normally. History is preserved.

If the commit is local only (not pushed): Use git reset --soft HEAD~1. The commit is removed. Changes are back in staging. No history rewriting on the remote.

> **Never git reset --hard on a pushed shared branch.** It rewrites history that others have. Their next pull breaks. Use revert for any pushed commit on a shared branch. This is non-negotiable.

Q67 Git bisect finds bug 200 commits ago

Step-by-Step

Step 1 - Start bisect. git bisect start. git bisect bad HEAD (current is broken). git bisect good v1.5.0 (this version was working). Git checks out the middle commit.

Step 2 - Test and mark. Run your test. git bisect good or git bisect bad. Git narrows the range and checks out the next middle commit. Repeat. 200 commits → found in ~8 steps (log2(200) ≈ 8).

Step 3 - Automate. git bisect run ./test.sh - Git runs the test script automatically at each step. Zero manual intervention. The script exits 0 for good, non-zero for bad.

Step 4 - Found the commit. Git identifies the exact commit that introduced the bug. Read the diff. Understand the change. Fix the bug with a new commit (do not modify the old one).

Step 5 - Clean up. git bisect reset returns to the original branch.

Q68 Submodule stuck on wrong commit

What Happened

A submodule points to an old commit. The parent repo was not updated after the submodule was updated. Deployments use the old version of the library.

Fix

cd into the submodule directory. git fetch origin. git checkout main (or the desired tag/commit). cd back to the parent repo. git add submodule-dir. git commit -m "Update submodule to latest version". git push.

Prevention

> CI should verify submodule versions match expectations. Add a pre-commit hook that warns when submodule pointers are outdated. Consider replacing submodules with a package manager (npm, pip) or Git subtree. Document the submodule update process prominently.

Q69 CI fails only on main - works on feature branches

Common Causes

Merge conflict resolution introduced a bug. The merge was resolved incorrectly - both sides' code is present but incompatible. **Environment differences.** Main deploys to staging with different config. Feature branches run in a lighter CI environment. **Cached dependencies.** Feature branch CI uses cached dependencies. Main CI has a stale or different cache. **Timing/flaky tests.** Tests pass individually but fail when run after other tests (shared state, database pollution).

Debugging

Check the merge commit diff - is the conflict resolution correct? Compare CI configs for main vs feature branches. Clear CI cache and re-run. Run the exact same test suite locally from the main branch.

Q70 Committed to wrong branch

Step-by-Step

Step 1 - If the commits are not pushed yet. Note the commit hashes. Switch to the correct branch: git switch correct-branch. Cherry-pick the commits: git cherry-pick abc1234 def5678. Switch back to the wrong branch: git switch wrong-branch. Reset: git reset --hard HEAD~2 (remove the 2 commits).

Step 2 - If committed to main and already pushed. git revert on main to undo the commits. Cherry-pick to the correct branch. Push both branches.

Step 3 - Shortcut if not pushed. git switch correct-branch, git merge wrong-branch (if you want all commits). Then git switch wrong-branch, git reset --hard origin/wrong-branch (restore the branch to remote state).

Q71 Deleted branch with unmerged work

Recovery

Step 1 - git reflog shows the branch tip before deletion. Find the last commit hash of the deleted branch.

Step 2 - Recreate the branch. git branch recovered-branch abc1234 (hash from reflog). All commits are restored.

Step 3 - If reflog does not show it (deleted on another machine or too long ago): git fsck --lost-found shows dangling commits. Search through them for your work.

> **Prevention:** git branch -d (lowercase d) refuses to delete unmerged branches. git branch -D (uppercase D) force deletes. Use -d by default - it protects you. Use -D only when you are certain.

Q72 200 tiny commits - clean before merge

Step-by-Step

Step 1 - Interactive rebase. git rebase -i main (or HEAD~200). Mark the first commit as pick. Mark the rest as squash or fixup. Save and exit. Write a clean, comprehensive commit message.

Step 2 - If too many to handle interactively, use squash merge instead: the PR merge strategy squashes all commits into one automatically. No manual rebase needed.

Step 3 - For partial squashing (keep 5 logical commits from 200): group related commits together with rebase -i. Squash each group. Reorder if needed.

> **Prevention:** Commit frequently (good) but squash before PR review (better). Use fixup commits: git commit --fixup=abc1234 marks a commit as a fix for an earlier one. git rebase -i --autosquash automatically orders and squashes fixup commits.

Q73 Two devs rebased same branch - duplicate commits

What Happened

Dev A and Dev B both work on shared-feature branch. Dev A rebases onto main and force pushes. Dev B pulls, but their local history has the old commit hashes. Dev B pushes - now the branch has duplicate commits (same changes, different hashes).

Fix

Step 1 - Decide on one version. Agree which developer's rebase is correct. **Step 2 - Reset to the correct version.** The other developer: git fetch origin, git reset --hard origin/shared-feature. **Step 3 - If commits are interleaved,** interactive rebase to remove duplicates.

Prevention

This is why the golden rule exists: never rebase shared branches. If rebasing is needed on a shared branch, coordinate: only one person rebases, force pushes, and notifies the team. Everyone else resets to the new remote. Or better: use merge instead of rebase on shared branches.

Q74 Hotfix needed but main has unreleased features

Step-by-Step

Step 1 - Create a hotfix branch from the last release tag, not from main. git switch -c hotfix/critical-bug v2.1.0. This ensures the hotfix only contains production code + the fix.

Step 2 - Make the fix, test, merge to production. Deploy from the hotfix branch (or tag it and deploy the tag).

Step 3 - Merge the hotfix back to main so the fix is included in future releases: git switch main, git merge hotfix/critical-bug.

Prevention

Always tag releases. Keep release branches if you support multiple versions. This workflow is core to Git Flow and is the standard for hotfix management.

Q75 .gitignore not working - file already tracked

Why It Happens

.gitignore only prevents untracked files from being tracked. If a file is ALREADY tracked (was committed before being added to .gitignore), .gitignore has no effect - Git continues tracking it.

Fix

git rm --cached filename - Remove the file from Git's index (stops tracking) without deleting it from the filesystem. git commit -m "Stop tracking filename". Now .gitignore takes effect for future changes.

For Multiple Files

git rm -r --cached . - Remove everything from the index. git add . - Re-add everything (respecting .gitignore). git commit -m "Apply .gitignore retrospectively". This is the nuclear option - works when you have many files to untrack.

Q76 Reverted merge commit, can't re-merge

What Happened

A feature branch was merged to main. A bug was found. The merge was reverted: git revert -m 1 merge_commit. Later, the bug was fixed on the feature branch. You try to merge the feature branch again - but Git says "Already up to date." The changes are not applied.

Why

Git's merge is based on commit ancestry, not file content. The feature branch's commits are already in main's history (via the original merge). The revert undid the changes but did not remove the commits from the history. Git thinks the merge is already done.

Fix

Option 1 - Revert the revert. git revert revert_commit_hash. This re-applies the original changes. Then merge the feature branch with the bug fix. Two revert commits in history is ugly but correct.

Option 2 - Rebase the feature branch. git rebase main on the feature branch creates new commits (new hashes). These are "new" commits from Git's perspective and can be merged normally.

> **This is one of the most confusing Git scenarios.** Interviewers love it because it tests deep understanding of how Git merge works - based on commit graph, not content comparison.

Q77 Large binary committed - repo bloated

What Happened

A developer committed a 2 GB video file. Even after deleting it in a subsequent commit, the repo is still 2 GB because Git stores every version of every file in history.

Fix

Step 1 - Remove from history. BFG: bfg --strip-blobs-bigger-than 50M. Or filter-repo: git filter-repo --strip-blobs-bigger-than 50M. **Step 2 - Force push.** git push --force --all. **Step 3 - All developers re-clone.** Old clones still contain the large file. **Step 4 - Run git gc --aggressive** on the server to reclaim space.

Prevention

> Pre-receive hook rejecting files over 5 MB. Set up Git LFS for binary files. Add binary patterns to .gitignore. Use .gitattributes to track large files with LFS: *.mp4 filter=lfs diff=lfs merge=lfs.

Q78 Git pull creates unexpected merge commits

What Happened

The developer's git log shows: "Merge branch 'main' of github.com/org/repo into main" - a merge commit that nobody intentionally created. History is polluted with meaningless merge commits.

Why

git pull (default) = fetch + merge. If the remote has commits the local does not (and vice versa), a merge commit is created to reconcile them. This happens naturally when multiple people push to the same branch.

Fix

Configure rebase as default: git config --global pull.rebase true. Now git pull = fetch + rebase. Local commits are replayed on top of remote commits. No merge commit. Linear history.

Or use explicitly: git pull --rebase origin main.

For the existing merge commits: Interactive rebase can remove them, but this rewrites history. For shared branches, accept the existing merge commits and prevent future ones with the config change.

Q79 Feature branch 3 months old, 500 behind main

Step-by-Step

Step 1 - Assess the branch. How many commits? How many files changed? Are the changes concentrated in a few files or spread across the codebase?

Step 2 - Try merging main into the branch first. git merge main. If conflicts are manageable (under 20 files), resolve them. This is the safest option - no history rewriting.

Step 3 - If conflicts are overwhelming, consider incremental rebasing. Instead of rebasing 500 commits at once, rebase onto an intermediate point (100 commits back from main), resolve conflicts, then rebase further. Git rerere helps - resolve each conflict once, rerere applies the resolution on subsequent encounters.

Step 4 - If the branch is too far gone, create a new branch from current main and manually reapply your changes. Use git diff main..feature-branch > changes.patch to get a patch of all changes. Apply selectively.

Prevention

Rebase onto main at least weekly. Ideal: daily. Maximum: bi-weekly. Beyond that, divergence makes integration exponentially harder. Trunk-based development eliminates this problem entirely - branches live for hours, not months.

Q80 Same conflict appearing repeatedly

What Happened

You rebase your feature branch onto main weekly. Each time, the same conflict appears in the same file. You resolve it, but next week it is back.

Why

Each rebase replays your commits one by one. If commit #3 in your branch conflicts with something in main, every rebase reproduces that conflict - even though you resolved it before. Rebase creates new commits (new hashes), so Git does not know the conflict was already resolved.

Fix

Enable git rerere. git config --global rerere.enabled true. Resolve the conflict once. Git records the resolution. On subsequent rebases, Git automatically applies the same resolution. No manual intervention.

Or squash before rebasing. If your 20 commits are squashed into 3, there are only 3 potential conflict points instead of 20. Fewer commits = fewer conflict opportunities.

rerere is one of Git's most underused features. It is designed exactly for this scenario - recurring conflicts in repeated rebase workflows. Enable it globally and forget about resolving the same conflict twice.

Interview Tips

For Junior Roles

Focus on Q1-Q25. Know the basic workflow (add, commit, push, pull). Understand branching and merging. Be able to explain rebase vs merge. Know how to resolve a merge conflict. Understand .gitignore and tags.

For Mid-Level Roles

Q1-Q48. Be ready to discuss branching strategies (and defend your team's choice). Know Git hooks, branch protection, and submodules. Understand squash vs merge commit. Have a real story about recovering from a Git mistake (reflog saved the day).

For Senior Roles

All 80 questions. Git internals, monorepo tooling, GitOps, compliance, and performance optimization separate senior from mid-level. Be ready to design a branching strategy for a team and discuss trade-offs.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Architecture	Working directory → staging → local repo → remote. Three trees model.
Branches	Cheap pointers. Create freely. Delete after merging. Name with prefixes.
Rebase vs merge	Rebase for personal branches (linear). Merge for shared branches (safe).
Golden rule	Never rebase a branch others are using. Never force push shared branches.
Reset	--soft (keep staged), --mixed (keep files), --hard (delete everything).
Revert vs reset	Revert for pushed commits (safe). Reset for local commits (rewrites history).
Reflog	Your safety net. Recovers "lost" commits for 90 days. git reflog.
.gitignore	Only affects untracked files. Already tracked? git rm --cached first.
Pull strategy	git pull --rebase avoids meaningless merge commits. Set as default.
Branching strategy	Trunk-based for speed. GitHub Flow for simplicity. Git Flow for releases.
Hooks	pre-commit for linting and secrets. Use pre-commit framework.
Branch protection	Required reviews + CI checks + no force push on main. Non-negotiable.
Submodules	Confusing. Consider subtree or package managers instead.
Monorepo	Sparse checkout + partial clone + affected-target CI for scale.
Signed commits	GPG or SSH signing. Required for compliance. "Verified" badge.
Secrets	Pre-commit scanning (gitleaks). Rotate immediately if committed.
LFS	For binary files. Track with .gitattributes. Reject large files in pre-receive.
rerere	Auto-resolve recurring conflicts. Enable globally. Underused superpower.
Git internals	Blob (content), tree (directory), commit (snapshot), tag (label). SHA-1 IDs.
GitOps	Git = source of truth. ArgoCD/Flux reconciles cluster to Git.
Cherry-pick	Apply specific commits. Different hashes. Use sparingly.
Bisect	Binary search for bugs. Automated with git bisect run. log2(n) steps.

Squash	Clean history before merge. fixup commits + autosquash for workflow.
Performance	Shallow clone for CI. fsmonitor for large repos. Maintenance for optimization.